

EJAVA Unit 1

Chapter 1 - Introducing Classes

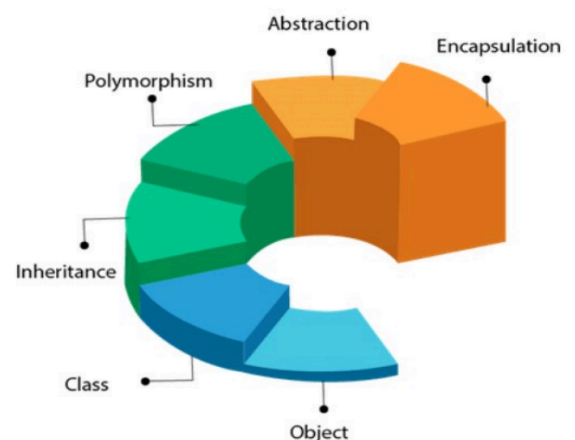
1.Object Oriented Programming System(OOPs)

Object Oriented Programming System (OOPs)

OOPs is a programming approach that organizes programs using objects and classes. It focuses on modeling real-world entities using data and behavior.

The main features of OOPs are:

- **Class** – blueprint for creating objects
- **Object** – instance of a class
- **Encapsulation** – wrapping data and methods into a single unit
- **Abstraction** – hiding implementation details and showing only essential features
- **Inheritance** – acquiring properties of one class into another
- **Polymorphism** – ability to perform one task in different ways



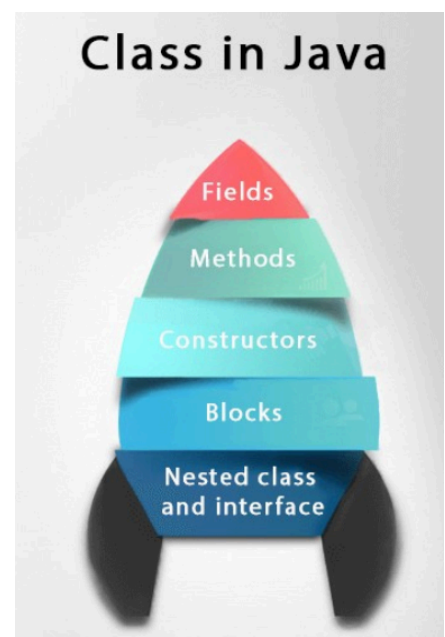
OOPs helps in code reusability, modularity, and easier maintenance of programs.

2.Class Fundamentals

A class is a blueprint for creating objects. It contains fields (variables) and methods (functions) that define the properties and behavior of objects.

Syntax to declare a class:

```
class ClassName {  
    field;  
    method;  
  
    public static void main(String args[]) {  
        // creating object  
        // accessing members  
    }  
}
```



An object is created using the **new** keyword:

```
ClassName obj = new ClassName();
```

Memory for the object is allocated at runtime in heap memory, and the reference variable points to that object.

Instance variable:

A variable declared inside a class but outside methods. It gets memory only when an object is created.

Method:

A function defined inside a class used to represent the behavior of an object. It provides code reusability and optimization.

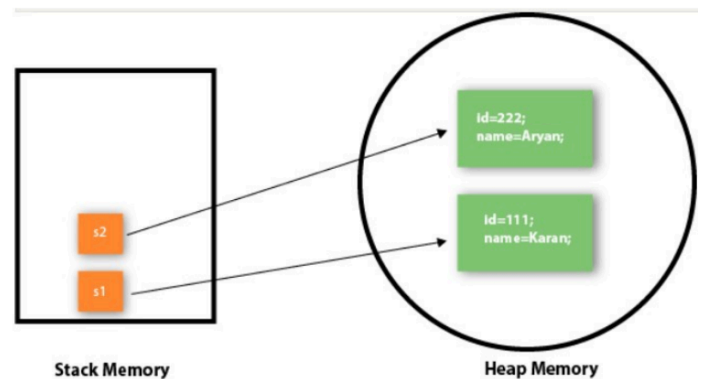
3.Ways to initialize object

Initialization means storing data into an object.

1. Initialization through reference

Values are assigned using reference variable after object creation.

```
class Student {  
    int id;  
    String name;  
  
    public static void main(String args[]) {  
        Student s1 = new Student();  
        s1.id = 101;  
        s1.name = "Shashidhara";  
    }  
}
```



2. Initialization through method

Values are assigned by calling a method.

```
class Student {  
    int id;  
    String name;  
  
    void insert(int i, String n) {  
        id = i;  
        name = n;  
    }  
}
```

```

}

public static void main(String args[]) {
    Student s1 = new Student();
    s1.insert(102, "Ram");
}
}

```

3. Initialization through constructor

Values are assigned using constructor at object creation.

```

class Student {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }

    public static void main(String args[]) {
        Student s1 = new Student(103, "Ravi");
    }
}

```

4. Rules for creating Java constructor

- Constructor name must be the same as the class name
- A constructor must not have any explicit return type
- A constructor cannot be abstract, static, final, or synchronized

5. Types of Java constructors

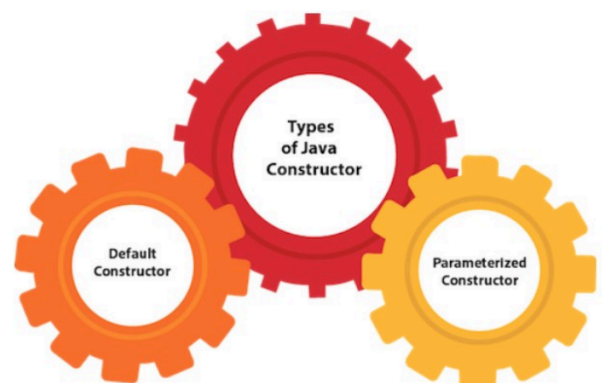
1. Default Constructor (No-argument constructor)

A constructor that does not have any parameters.

```

class Demo {
    Demo() {
        System.out.println("Default Constructor");
    }
}

```



```

public static void main(String args[]) {
    Demo d = new Demo();
}
}

```

2. Parameterized Constructor

A constructor that has parameters to initialize objects with specific values.

```

class Demo {
    int id;
    String name;

    Demo(int i, String n) {
        id = i;
        name = n;
    }

    public static void main(String args[]) {
        Demo d = new Demo(101, "ABC");
    }
}

```

6.Constructor & Method Overloading

Constructor Overloading

Constructor overloading is a technique of having more than one constructor in a class with different parameter lists (different number or type of parameters). Each constructor performs a different task.

```

class Demo {
    Demo() {
        System.out.println("Default");
    }

    Demo(int a) {
        System.out.println(a);
    }

    public static void main(String args[]) {
        Demo d1 = new Demo();
        Demo d2 = new Demo(10);
    }
}

```

Method Overloading

Method overloading means defining multiple methods with the same name but different parameters. It improves readability of the program.

```
class Demo {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static void main(String args[]) {  
        Demo d = new Demo();  
        System.out.println(d.add(2, 3));  
        System.out.println(d.add(2, 3, 4));  
    }  
}
```

Method overloading is not possible by changing only the return type due to ambiguity.

7. Why Method Overloading is not possible by changing the return type of method only?

Method overloading is not possible by changing only the return type because it leads to ambiguity. The compiler cannot differentiate between methods if they have the same name and parameters but different return types.

This results in a compile-time error since the method call does not specify which return type to use.

8. Can you overload java main() method?

Yes, the `main()` method can be overloaded in Java by defining multiple methods with the same name but different parameter lists.

However, the JVM calls only the `main()` method with the signature:

```
public static void main(String[] args)
```

Other overloaded `main()` methods are not executed automatically.

9. Can You overload java main() method?

Yes, you can overload the `main()` method in Java by using different parameter lists.

However, JVM executes only the method with the signature:

```
public static void main(String[] args)
```

Other overloaded `main()` methods will not be called automatically.

10. Differences between Constructor and Method

Constructor	Method
Used to initialize objects	Used to define behavior of objects
Name must be the same as the class name	Name can be any valid identifier
Does not have any return type (not even void)	Must have a return type or void
Called automatically when an object is created	Called explicitly using object or class
Invoked only once at the time of object creation	Can be invoked multiple times
Used mainly for initializing instance variables	Used to perform operations or computations
Cannot be inherited	Can be inherited
Cannot be overridden	Can be overridden
Can be overloaded	Can be overloaded
Default constructor is provided by compiler if no constructor is defined	No default method is provided automatically
Memory allocation for object happens when constructor is called	No role in memory allocation
Can use access modifiers (public, private, protected, default)	Can also use access modifiers
Cannot be static, abstract, final, or synchronized	Methods can be static, abstract, final, or synchronized

11. this Keyword

this is a reference variable that refers to the current object.

Uses of **this** keyword:

- Used to refer to current class instance variables
- Used to invoke current class methods
- Used to invoke current class constructor using **this()**
- Can be passed as an argument in a method call
- Can be passed as an argument in a constructor call
- Can be used to return the current class instance
- Used to differentiate instance variables and local variables when names are same

Example:

```
class Demo {
    int id;
    String name;

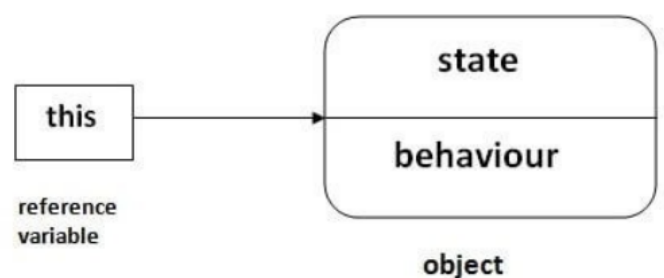
    // 3) this() to invoke constructor (constructor chaining)
    Demo() {
        this(100, "ABC"); // calls parameterized constructor
    }

    Demo(int id, String name) {
        // 1) this to refer current class instance variable
        this.id = id;
        this.name = name;
    }

    // 2) this to invoke current class method
    void display() {
        this.show();
    }

    void show() {
        System.out.println(id + " " + name);
    }

    // 4) this as argument in method
    void get(Demo d) {
        System.out.println("Method called");
    }
}
```



```

    }

    void passMethod() {
        get(this);
    }

    // 5) this as argument in constructor
    Demo(Demo d) {
        System.out.println("Constructor called");
    }

    void passConstructor() {
        new Demo(this);
    }

    // 6) this to return current class instance
    Demo returnThis() {
        return this;
    }

    public static void main(String args[]) {
        Demo d = new Demo(); // constructor chaining
        d.display();        // method call using this
        d.passMethod();      // passing this to method
        d.passConstructor(); // passing this to constructor
        d.returnThis().display(); // returning current object
    }
}

```

12. Introducing Access Control

Access control in Java is used to define the accessibility (scope) of classes, fields, methods, and constructors. It is achieved using access modifiers.

There are two types of modifiers:

- **Access Modifiers**
- **Non-Access Modifiers**

Access Modifiers in Java:

- **Default** – accessible within the same package
- **Private** – accessible only within the same class
- **Protected** – accessible within the same package and in subclasses

- **Public** – accessible from anywhere

Non-Access Modifiers:

static, abstract, synchronized, native, volatile, transient, etc.

Access modifiers help in achieving data security and control over visibility of class members.

13. Access Modifiers

Access modifiers in Java define the scope and accessibility of classes, variables, methods, and constructors.

There are four types of access modifiers:

- **Private**
Accessible only within the same class.
- **Default (Package-private)**
Accessible only within the same package.
- **Protected**
Accessible within the same package and also in subclasses (even in different packages).
- **Public**
Accessible from anywhere.

Access modifiers are used to control visibility and provide data security.

Modifier	Same Class	Same Package	Subclass (Different Package)	Outside Package
Private	Yes	No	No	No
Default	Yes	Yes	No	No
Protected	Yes	Yes	Yes	No
Public	Yes	Yes	Yes	Yes

14. Private Access Modifier within Class/Package

- The **private** access modifier allows access only within the same class.
- It cannot be accessed from outside the class, even within the same package.

Example:

```
class Demo {
    private int data = 10;

    void display() {
        System.out.println(data); // accessible within same class
    }
}

class Test {
    public static void main(String args[]) {
        Demo d = new Demo();
        // System.out.println(d.data); // Error: not accessible
    }
}
```

15. Default / Public / Protected Access Modifiers within Class/Package

- **Default (no modifier)**
Accessible within the same class and same package only.
- **Protected**
Accessible within the same class, same package, and also in subclasses (even in different packages).
- **Public**
Accessible from anywhere (same class, same package, different package).

Example:

```
class Demo {
    int a = 10;      // default
    protected int b = 20;
    public int c = 30;

    void display() {
        System.out.println(a + " " + b + " " + c); // all accessible
    }
}

class Test {
    public static void main(String args[]) {
        Demo d = new Demo();
    }
}
```

```

        System.out.println(d.a); // default - accessible (same package)
        System.out.println(d.b); // protected - accessible (same package)
        System.out.println(d.c); // public - accessible
    }
}

```

16. Understanding Static

The **static** keyword in Java is used mainly for memory management. It is associated with the class rather than objects.

- Static members belong to the class, not to any instance
- They can be variables, methods, blocks, or nested classes
- Static members can be accessed without creating an object
- Used for common properties shared by all objects

Example:

```

class Demo {
    static int count = 0; // static variable

    Demo() {
        count++;
    }

    public static void main(String args[]) {
        new Demo();
        new Demo();
        System.out.println(Demo.count); // accessed using class name
    }
}

```

17. Characteristics of static keyword

- Static members are allocated memory only once during program execution and shared among all objects
- They can be accessed without creating an object
- Static members belong to the class, not to individual objects
- Changes made to static variables are reflected in all instances
- Static methods can access only static data and call only static methods directly
- Static methods cannot use **this** or **super**
- Static methods can be overloaded but cannot be overridden

18. Member as Static

When a member is declared as **static**, it can be accessed before creating any object of the class and without referring to any object.

Static members belong to the class rather than instances and are usually accessed using the class name.

Example:

```
class Test {
    static void m1() {
        System.out.println("Static method called");
    }

    public static void main(String args[]) {
        Test.m1(); // accessing without creating object
    }
}
```

19. Static Blocks

A static block is used to initialize static variables. It is executed only once when the class is loaded into memory.

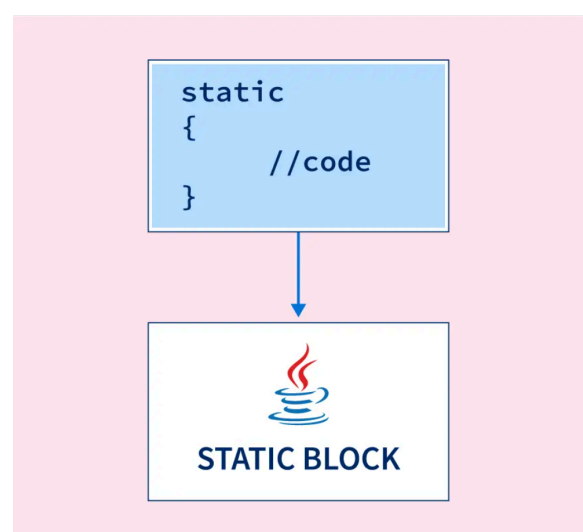
- Executed before the **main()** method
- Used for complex initialization of static data

Example:

```
class Demo {
    static int a;

    static {
        a = 10;
        System.out.println("Static block
executed");
    }

    public static void main(String args[]) {
        System.out.println(a);
    }
}
```



20. Static Variables

A static variable is a variable declared with the **static** keyword. It is shared among all objects of the class and only one copy is created at the class level.

- Memory is allocated only once and shared by all instances
- It is also known as a class variable
- All objects share the same value
- Declared at class level

Example:

```
class Demo {
    static int count = 0;

    Demo() {
        count++;
        System.out.println(count);
    }

    public static void main(String args[]) {
        new Demo();
        new Demo();
        new Demo();
    }
}
```

21. Static Methods

A method declared with the **static** keyword is called a static method. It belongs to the class rather than an object.

- Can be called without creating an object
- The **main()** method is a common example
- Can directly access only static data
- Can directly call only static methods
- Cannot use **this** or **super**

Example:

```
class Demo {
    static int a = 10;

    static void show() {
        System.out.println(a); // accessing static variable
    }
}
```

```

    }

    public static void main(String args[]) {
        Demo.show(); // calling static method without object
    }
}

```

22. When to use static variables and methods

- Use **static variables** for properties that are common to all objects of a class (shared data)
- Use **static methods** to access or modify static variables and perform operations not dependent on object data

Example:

```

class Student {
    int id;
    String name;
    static String college = "ABC College"; // common for all students

    Student(int i, String n) {
        id = i;
        name = n;
    }

    static void changeCollege() {
        college = "XYZ College"; // modifying static variable
    }

    void display() {
        System.out.println(id + " " + name + " " + college);
    }

    public static void main(String args[]) {
        Student.changeCollege(); // calling static method

        Student s1 = new Student(1, "A");
        Student s2 = new Student(2, "B");

        s1.display();
        s2.display();
    }
}

```

23. Nested Classes

A nested class is a class defined inside another class. It is used to logically group classes and improve encapsulation and readability.

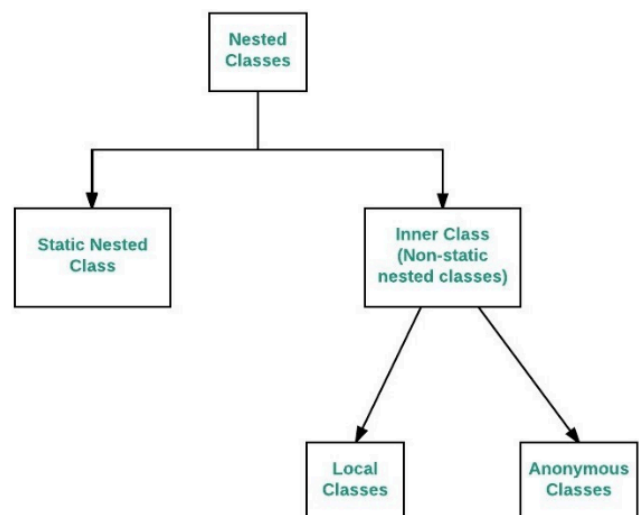
- Scope of nested class is limited to its enclosing class
- It can access members (including private) of the outer class
- Outer class cannot directly access members of nested class
- Nested class is treated as a member of the outer class
- It can have access modifiers (private, public, protected, default)

Types of nested classes:

- **Static nested class**
- **Inner class (non-static nested class)**

Example:

```
class Outer {  
    int x = 10;  
  
    class Inner {  
        void display() {  
            System.out.println(x); // accessing outer class member  
        }  
    }  
}  
  
public static void main(String args[]) {  
    Outer obj = new Outer();  
    Outer.Inner in = obj.new Inner();  
    in.display();  
}
```



24. Static Nested Classes

A static nested class is a class declared with the **static** keyword inside another class. It is associated with the outer class, not with its objects.

- Can be created without creating an object of the outer class

- Accessed using the outer class name
- Cannot directly access non-static members of the outer class
- Can access static members of the outer class
- Not strongly associated with outer class objects

Example:

```
class Outer {
    static int x = 10;

    static class Inner {
        void display() {
            System.out.println(x); // accessing static member
        }
    }

    public static void main(String args[]) {
        Outer.Inner obj = new Outer.Inner(); // no outer object needed
        obj.display();
    }
}
```

25. Inner Classes

An inner class is a non-static class defined inside another class. It is associated with an instance of the outer class.

- Requires an object of the outer class to create an inner class object
- Can access all members (including private) of the outer class
- Strongly associated with the outer class object

Syntax to create inner class object:

```
OuterClass.InnerClass obj = outerObject.new InnerClass();
```

Example:

```
class Outer {
    int x = 10;

    class Inner {
        void display() {
            System.out.println(x); // accessing outer class member
        }
    }
}
```

```
class Outer_class {
    class Inner_class {
        //code
    }
}
```



```

    }
}

public static void main(String args[]) {
    Outer o = new Outer();
    Outer.Inner i = o.new Inner(); // creating inner class object
    i.display();
}
}

```

26. Comparison between Normal (Regular) Inner Class and Static Nested Class

Normal / Regular Inner Class	Static Nested Class
Non-static nested class	Declared with static keyword
Requires outer class object to create object	Does not require outer class object
Strongly associated with outer class instance	Not strongly associated with outer class
Can access both static and non-static members directly	Can access only static members directly
Can access instance members without object reference	Needs outer class object to access instance members
Cannot define static members (except constants)	Can define static members
Created using outerObj.new Inner()	Created using Outer.Inner obj = new Outer.Inner()
Exists only when outer object exists	Can exist independently
Used when inner class needs frequent access to outer instance data	Used when functionality is related to outer class but independent of instance
More memory overhead due to outer instance reference	Less memory overhead (no outer instance reference)

27. String Class

String is a sequence of characters. In Java, String is an object that represents a sequence of characters and belongs to `java.lang.String` class.

Ways to create String objects:

1. **Using String literal**

- Stored in String Constant Pool
- If the same string exists, reference to existing object is returned

```
String s1 = "Welcome";
```

```
String s2 = "Welcome"; // refers to same object
```

2. **Using new keyword**

- Creates a new object in heap memory
- Literal is stored in String Constant Pool

```
String s = new String("Welcome");
```

- String objects are stored in a special memory area called **String Constant Pool**
- Strings are immutable (cannot be changed once created)

28. String Class Methods

Method	Description
<code>length()</code>	Returns length of the string
<code>charAt(int index)</code>	Returns character at given index
<code>substring(int begin, int end)</code>	Returns substring
<code>equals(Object obj)</code>	Compares content of strings
<code>equalsIgnoreCase(String s)</code>	Compares ignoring case
<code>toLowerCase()</code>	Converts string to lowercase
<code>toUpperCase()</code>	Converts string to uppercase
<code>trim()</code>	Removes leading and trailing spaces
<code>replace(char old, char new)</code>	Replaces characters

<code>contains(CharSequence s)</code>	Checks if substring exists
<code>startsWith(String prefix)</code>	Checks starting characters
<code>endsWith(String suffix)</code>	Checks ending characters
<code>indexOf(String s)</code>	Returns index of substring
<code>lastIndexOf(String s)</code>	Returns last occurrence index
<code>concat(String s)</code>	Concatenates strings

Example:

```
class Demo {
    public static void main(String args[]) {
        String s = " Hello Java ";

        System.out.println(s.length());
        System.out.println(s.trim());
        System.out.println(s.toUpperCase());
        System.out.println(s.substring(1, 5));
        System.out.println(s.contains("Java"));
    }
}
```

29. StringBuilder Class

StringBuilder is a class used to create **mutable (modifiable) strings** in Java.

- Unlike String, objects of StringBuilder can be changed after creation
- It is more efficient when performing multiple string modifications
- Located in `java.lang` package

Example:

```
class Demo {
    public static void main(String args[]) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" Java"); // modifying string
        System.out.println(sb);
    }
}
```

30. Methods of StringBuilder Class

Method	Description
<code>append(String s)</code>	Adds text at the end
<code>insert(int offset, String s)</code>	Inserts text at specified position
<code>replace(int start, int end, String s)</code>	Replaces characters in a range
<code>delete(int start, int end)</code>	Deletes characters in a range
<code>reverse()</code>	Reverses the string
<code>charAt(int index)</code>	Returns character at index
<code>setCharAt(int index, char ch)</code>	Changes character at index
<code>length()</code>	Returns length of sequence
<code>capacity()</code>	Returns current capacity
<code>ensureCapacity(int min)</code>	Ensures minimum capacity
<code>substring(int start)</code>	Returns substring from index
<code>substring(int start, int end)</code>	Returns substring in range

Example:

```
class Demo {  
    public static void main(String args[]) {  
        StringBuilder sb = new StringBuilder("Hello");  
  
        sb.append(" Java");      // append  
        sb.insert(5, " World");  // insert  
        sb.replace(0, 5, "Hi");  // replace  
        sb.delete(2, 7);         // delete  
        sb.reverse();            // reverse  
  
        System.out.println(sb);  // final string  
        System.out.println(sb.charAt(2)); // charAt  
        sb.setCharAt(2, 'X');    // setCharAt  
  
        System.out.println(sb.length()); // length  
        System.out.println(sb.capacity()); // capacity  
    }  
}
```

```

        System.out.println(sb.substring(1));    // substring
        System.out.println(sb.substring(1, 4)); // substring range
    }
}

```

31. Java I/O Basics

Java I/O (Input/Output) is used to handle data input and output operations such as reading from and writing to files, streams, or other sources.

- Java uses **streams** to perform I/O operations
- A **stream** is a sequence of data (bytes or characters)

Types of Streams:

- **Byte Streams** – handle data in bytes (e.g., images, files)
- **Character Streams** – handle data in characters (text data)

Basic I/O Classes:

- **InputStream** and **OutputStream** (byte streams)
- **Reader** and **Writer** (character streams)

Example:

```

import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        FileOutputStream fout = new FileOutputStream("test.txt");
        fout.write(65); // writes 'A'
        fout.close();

        FileInputStream fin = new FileInputStream("test.txt");
        int i = fin.read();
        System.out.println((char)i); // reads 'A'
        fin.close();
    }
}

```

32. Java I/O Streams

A stream is a sequence of data used for input and output operations in Java. It allows reading from and writing to different sources like files, keyboard, or network.

Types of Streams:

1. **Byte Streams**
 - Handle data in the form of bytes
 - Used for binary data (images, audio, files)
 - Classes: **InputStream**, **OutputStream**
2. **Character Streams**
 - Handle data in the form of characters
 - Used for text data
 - Classes: **Reader**, **Writer**

Key Points:

- Input streams are used to read data
- Output streams are used to write data
- Streams provide a uniform way to handle I/O operations

Example:

```
import java.io.*;

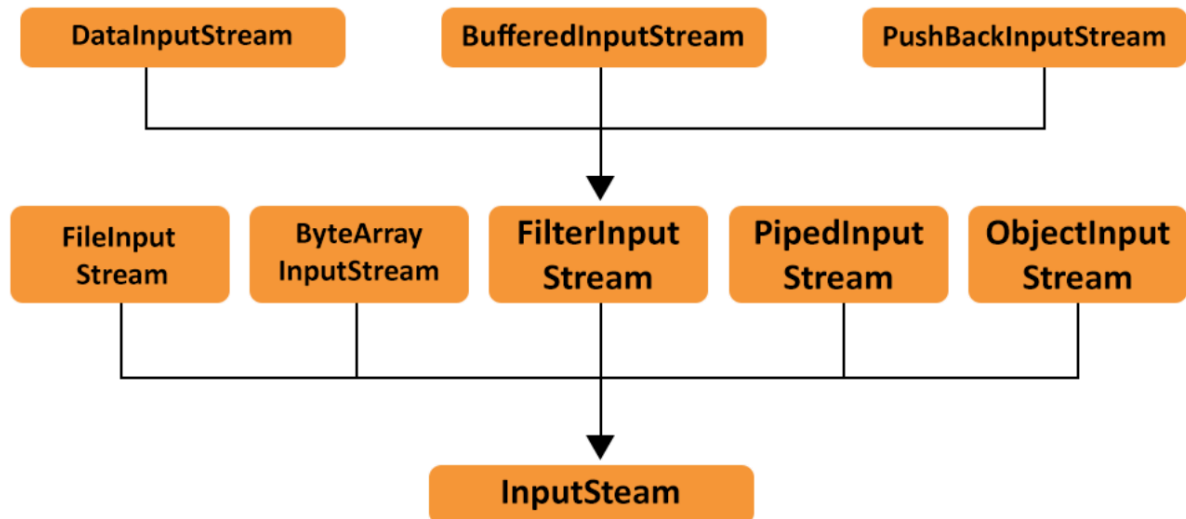
class Demo {
    public static void main(String args[]) throws Exception {
        FileWriter fw = new FileWriter("test.txt");
        fw.write("Hello");
        fw.close();

        FileReader fr = new FileReader("test.txt");
        int i;
        while((i = fr.read()) != -1) {
            System.out.print((char)i);
        }
        fr.close();
    }
}
```

33. Java InputStream

InputStream is an abstract class used to read data in the form of bytes from a source such as a file, keyboard, or network.

- It belongs to **java.io** package
- Used for reading binary data
- It is the superclass of all byte input streams



Common Methods:

- `int read()` – reads a single byte
- `int read(byte[] b)` – reads bytes into array
- `void close()` – closes the stream

Example:

```
import java.io.*;
```

```
class Demo {
    public static void main(String args[]) throws Exception {
        FileInputStream fin = new FileInputStream("test.txt");

        int i;
        while((i = fin.read()) != -1) {
            System.out.print((char)i);
        }

        fin.close();
    }
}
```

34. Java OutputStream

`OutputStream` is an abstract class used to write data in the form of bytes to a destination such as a file, console, or network.

- Belongs to `java.io` package
- Used for writing binary data
- It is the superclass of all byte output streams

- Requires `InputStreamReader`

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter name: ");
        String name = br.readLine(); // reads string

        System.out.println("Name: " + name);
    }
}
```

Using Scanner:

- Easier way to take input

```
import java.util.*;

class Demo {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number: ");
        int n = sc.nextInt();

        System.out.println("Number: " + n);
    }
}
```

36. Reading using Console Class

The `Console` class is used to read input from the console. It is available in the `java.io` package.

- Obtained using `System.console()`
- Used to read text and password input
- Password input is more secure (not displayed on screen)

Common Methods:

- `readLine()` – reads a string
- `readPassword()` – reads password as character array

Example:

```
import java.io.*;

class Demo {
    public static void main(String args[]) {
        Console c = System.console();

        String name = c.readLine("Enter name: ");
        char[] pwd = c.readPassword("Enter password: ");

        System.out.println("Name: " + name);
        System.out.println("Password: " + String.valueOf(pwd));
    }
}
```

37. Writing Console Output

In Java, console output is displayed using output streams like `System.out`, which is a `PrintStream` object.

Common Methods:

- `print()` – prints data without new line
- `println()` – prints data with new line
- `printf()` – prints formatted output

Example:

```
class Demo {
    public static void main(String args[]) {
        int a = 10;
        String name = "Java";

        System.out.print("Value: " + a); // no new line
        System.out.println();           // new line
        System.out.println("Name: " + name); // with new line

        System.out.printf("Value = %d", a); // formatted output
    }
}
```

38. Writing Console Output using write()

In Java, output can also be written to the console using the `write()` method of `OutputStream` (`System.out`).

- `write(int b)` writes a single byte (character)
- It outputs ASCII value as character

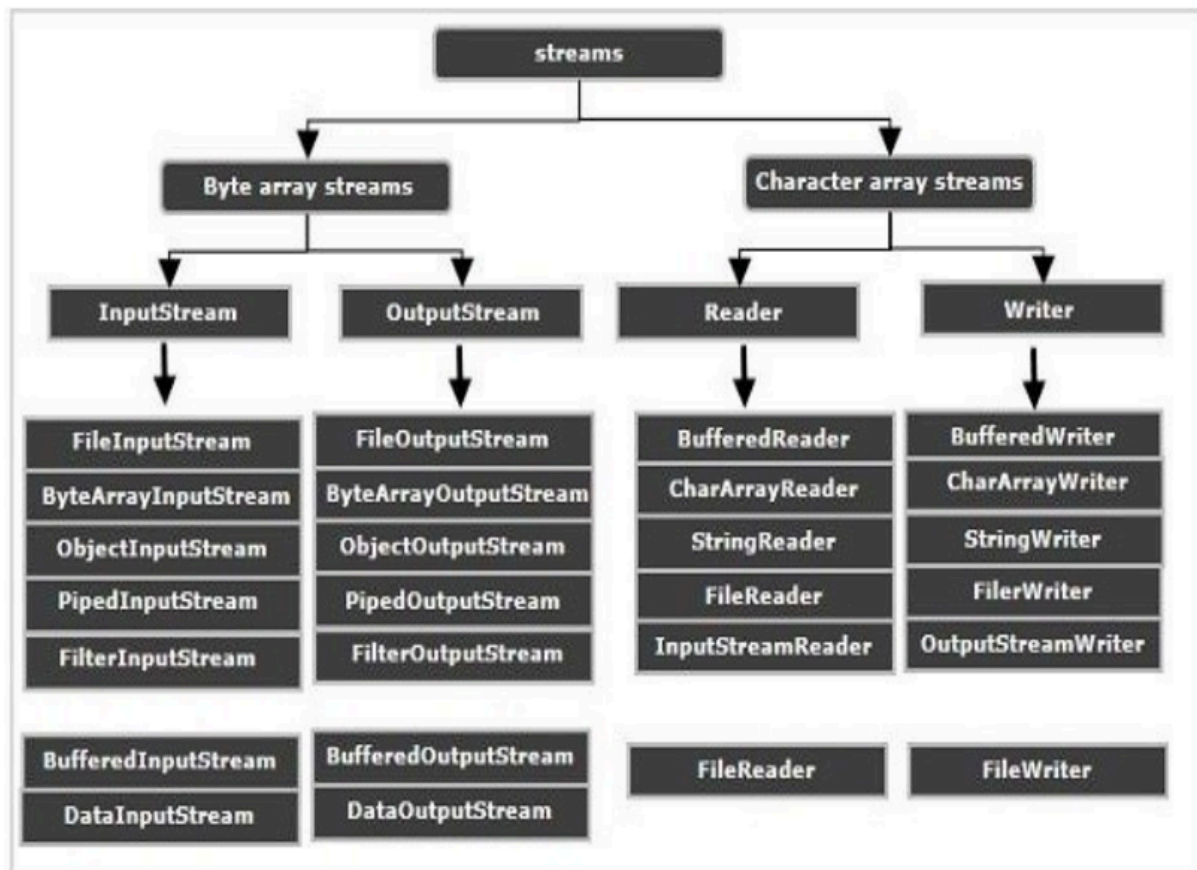
Example:

```
class Demo {  
    public static void main(String args[]) throws Exception {  
        System.out.write(65); // ASCII value of 'A'  
        System.out.write('\n'); // new line  
    }  
}
```

39. Types of Streams

In Java, streams are classified into two types:

1. **Byte Streams**
 - Handle data in the form of bytes
 - Used for binary data (images, audio, files)
 - Base classes: `InputStream`, `OutputStream`
2. **Character Streams**
 - Handle data in the form of characters
 - Used for text data
 - Base classes: `Reader`, `Writer`



40. Byte Streams – FileInputStream

FileInputStream is a class used to read data from a file in the form of bytes. It is a subclass of **InputStream** and belongs to **java.io** package.

- Used to read binary data from files
- Reads one byte at a time
- Commonly used for files like images, audio, etc.

Common Methods:

- **int read()** – reads a byte
- **int read(byte[] b)** – reads bytes into array
- **void close()** – closes the stream

Example:

```
import java.io.*;
```

```
class Demo {
    public static void main(String args[]) throws Exception {
        FileInputStream fin = new FileInputStream("test.txt");
```

```

int i;
while((i = fin.read()) != -1) {
    System.out.print((char)i);
}

fin.close();
}
}

```

41. Methods of FileInputStream Class

Method	Description
<code>int read()</code>	Reads a single byte from the file
<code>int read(byte[] b)</code>	Reads bytes into an array
<code>int available()</code>	Returns number of bytes available to read
<code>long skip(long n)</code>	Skips specified number of bytes
<code>void close()</code>	Closes the stream

42. FileOutputStream

`FileOutputStream` is a class used to write data to a file in the form of bytes. It is a subclass of `OutputStream` and belongs to `java.io` package.

- Used to write binary data to files
- Writes one byte at a time or in byte arrays
- Creates file if it does not exist

Common Methods:

- `void write(int b)` – writes a single byte
- `void write(byte[] b)` – writes byte array
- `void close()` – closes the stream

Example:

```

import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {

```

```

        FileOutputStream fout = new FileOutputStream("test.txt");

        String s = "Hello Java";
        byte b[] = s.getBytes();

        fout.write(b); // writing bytes
        fout.close();
    }
}

```

43. DataInputStream Class

DataInputStream is used to read primitive data types (int, float, double, etc.) and strings from an input stream in a machine-independent way. It is a subclass of **FilterInputStream** and belongs to **java.io** package.

- Used along with other input streams (like **FileInputStream**)
- Reads data in a structured format
- Supports reading different data types

Common Methods:

- **readInt()** – reads an integer
- **readFloat()** – reads a float
- **readDouble()** – reads a double
- **readChar()** – reads a character
- **readUTF()** – reads a string
- **close()** – closes the stream

Example:

```

import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        FileInputStream fin = new FileInputStream("test.txt");
        DataInputStream din = new DataInputStream(fin);

        System.out.println(din.readInt()); // reading integer

        din.close();
        fin.close();
    }
}

```

44. DataOutputStream Class

`DataOutputStream` is used to write primitive data types (int, float, double, etc.) and strings to an output stream in a machine-independent way. It is a subclass of `FilterOutputStream` and belongs to `java.io` package.

- Used along with other output streams (like `FileOutputStream`)
- Writes data in a structured format
- Supports writing different data types

Common Methods:

- `writeInt(int v)` – writes an integer
- `writeFloat(float v)` – writes a float
- `writeDouble(double v)` – writes a double
- `writeChar(int v)` – writes a character
- `writeUTF(String s)` – writes a string
- `close()` – closes the stream

Example:

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        FileOutputStream fout = new FileOutputStream("test.txt");
        DataOutputStream dout = new DataOutputStream(fout);

        dout.writeInt(100);
        dout.writeUTF("Hello");

        dout.close();
        fout.close();
    }
}
```

45. RandomAccessFile Class

`RandomAccessFile` is used to read and write data from a file at any position (randomly). It belongs to the `java.io` package.

- Allows both reading and writing operations
- File pointer can be moved to any position
- Supports random access (not sequential)

Modes:

- "r" – read only
- "rw" – read and write

Common Methods:

- seek(long pos) – moves file pointer to specified position
- read() – reads a byte
- write(int b) – writes a byte
- getFilePointer() – returns current position
- close() – closes the file

Example:

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        RandomAccessFile file = new RandomAccessFile("test.txt", "rw");

        file.writeBytes("Hello World");
        file.seek(6); // move pointer
        file.writeBytes("Java");

        file.seek(0);
        String line = file.readLine();
        System.out.println(line);

        file.close();
    }
}
```

46. Character Streams

Character streams are used to handle data in the form of characters (text data) in Java.

- Used for reading and writing text data
- Handle Unicode characters
- More suitable for character-based data

Base Classes:

- Reader – for reading characters
- Writer – for writing characters

Common Classes:

- `FileReader`
- `FileWriter`
- `BufferedReader`
- `BufferedWriter`

Example:

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        FileWriter fw = new FileWriter("test.txt");
        fw.write("Hello Java");
        fw.close();

        FileReader fr = new FileReader("test.txt");
        int i;
        while((i = fr.read()) != -1) {
            System.out.print((char)i);
        }
        fr.close();
    }
}
```

47. Java Reader

`Reader` is an abstract class used to read character streams (text data) in Java. It belongs to the `java.io` package.

- Used for reading characters instead of bytes
- Base class for all character input streams
- Supports Unicode characters

Common Methods:

- `int read()` – reads a single character
- `int read(char[] cbuf)` – reads characters into array
- `void close()` – closes the stream

Common Subclasses:

- `FileReader`
- `BufferedReader`

- `InputStreamReader`

Example:

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        Reader r = new FileReader("test.txt");

        int i;
        while((i = r.read()) != -1) {
            System.out.print((char)i);
        }

        r.close();
    }
}
```

48. Java Writer

`Writer` is an abstract class used to write character streams (text data) in Java. It belongs to the `java.io` package.

- Used for writing characters instead of bytes
- Base class for all character output streams
- Supports Unicode characters

Common Methods:

- `void write(int c)` – writes a single character
- `void write(char[] cbuf)` – writes character array
- `void write(String str)` – writes a string
- `void close()` – closes the stream

Common Subclasses:

- `FileWriter`
- `BufferedWriter`
- `OutputStreamWriter`

Example:

```
import java.io.*;
```

```

class Demo {
    public static void main(String args[]) throws Exception {
        Writer w = new FileWriter("test.txt");

        w.write("Hello Java"); // writing string
        w.close();
    }
}

```

49. Java Reader & Writer

Reader

- Abstract class used to read character streams (text data)
- Reads data in the form of characters
- Base class for character input streams
- Common methods: `read()`, `read(char[] cbuf)`, `close()`
- Subclasses: `FileReader`, `BufferedReader`, `InputStreamReader`

Writer

- Abstract class used to write character streams (text data)
- Writes data in the form of characters
- Base class for character output streams
- Common methods: `write(int c)`, `write(char[] cbuf)`, `write(String str)`, `close()`
- Subclasses: `FileWriter`, `BufferedWriter`, `OutputStreamWriter`

Example:

```

import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {
        Writer w = new FileWriter("test.txt");
        w.write("Hello Java");
        w.close();

        Reader r = new FileReader("test.txt");
        int i;
        while((i = r.read()) != -1) {
            System.out.print((char)i);
        }
        r.close();
    }
}

```

50. Java FileReader and FileWriter Class

FileReader

- Used to read character data from a file
- Subclass of `Reader`
- Suitable for reading text files
- Reads data character by character

Common Methods:

- `int read()` – reads a character
- `int read(char[] cbuf)` – reads characters into array
- `void close()` – closes the stream

FileWriter

- Used to write character data to a file
- Subclass of `Writer`
- Suitable for writing text data
- Creates file if it does not exist

Common Methods:

- `void write(int c)` – writes a character
- `void write(char[] cbuf)` – writes character array
- `void write(String str)` – writes a string
- `void close()` – closes the stream

Example:

```
import java.io.*;
```

```
class Demo {  
    public static void main(String args[]) throws Exception {  
        FileWriter fw = new FileWriter("test.txt");  
        fw.write("Hello Java");  
        fw.close();  
  
        FileReader fr = new FileReader("test.txt");  
        int i;  
        while((i = fr.read()) != -1) {  
            System.out.print((char)i);  
        }  
    }  
}
```

```
        fr.close();
    }
}
```

51. Java BufferedReader and BufferedWriter Class

BufferedReader

- Used to read text efficiently from a character input stream
- Subclass of **Reader**
- Reads data line by line
- Improves performance using buffering

Common Methods:

- **readLine()** – reads a line of text
- **read()** – reads a character
- **close()** – closes the stream

BufferedWriter

- Used to write text efficiently to a character output stream
- Subclass of **Writer**
- Uses buffer to improve performance

Common Methods:

- **write(String s)** – writes a string
- **newLine()** – writes a new line
- **close()** – closes the stream

Example:

```
import java.io.*;
```

```
class Demo {
    public static void main(String args[]) throws Exception {
        BufferedWriter bw = new BufferedWriter(new FileWriter("test.txt"));
        bw.write("Hello Java");
        bw.newLine();
        bw.write("Buffered Writer");
        bw.close();

        BufferedReader br = new BufferedReader(new FileReader("test.txt"));
        String line;
```

```

        while((line = br.readLine()) != null) {
            System.out.println(line);
        }
        br.close();
    }
}

```

52. Java Serialization

Serialization is the process of converting an object into a byte stream so that it can be stored in a file or transmitted over a network.

- Used for saving object state
- Helps in data transfer between systems
- Achieved using **Serializable** interface

Deserialization

The reverse process of converting byte stream back into an object.

Classes Used:

- **ObjectOutputStream** – to write object
- **ObjectInputStream** – to read object

Example:

```

import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }
}

class Demo {
    public static void main(String args[]) throws Exception {
        // Serialization
        ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("test.txt"));
        oos.writeObject(new Student(1, "A"));
    }
}

```

```

        oos.close();

        // Deserialization
        ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("test.txt"));
        Student s = (Student) ois.readObject();
        System.out.println(s.id + " " + s.name);
        ois.close();
    }
}

```

53. Java Serialization – java.io.Serializable Interface

Serializable is a marker interface used to enable serialization of objects in Java. It belongs to the **java.io** package.

- It does not contain any methods (marker interface)
- A class must implement **Serializable** to convert its objects into byte stream
- If a class does not implement it, its objects cannot be serialized
- Used with **ObjectOutputStream** and **ObjectInputStream**

Example:

```

import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }
}

class Demo {
    public static void main(String args[]) throws Exception {
        ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("test.txt"));
        oos.writeObject(new Student(1, "A")); // serialization
        oos.close();
    }
}

```

54. Java Serialization – ObjectOutputStream Class

ObjectOutputStream is used to write objects to an output stream (serialize objects into byte stream). It belongs to the **java.io** package.

- Used for serialization of objects
- Writes objects to a file or stream
- Works with classes that implement **Serializable**

Common Methods:

- **writeObject(Object obj)** – writes object to stream
- **flush()** – clears buffer and writes data
- **close()** – closes the stream

Example:

```
import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }
}

class Demo {
    public static void main(String args[]) throws Exception {
        FileOutputStream fout = new FileOutputStream("test.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fout);

        Student s = new Student(1, "A");
        oos.writeObject(s); // writing object

        oos.close();
        fout.close();
    }
}
```


55. Java Serialization – ObjectInputStream Class

`ObjectInputStream` is used to read objects from an input stream (deserialization). It belongs to the `java.io` package.

- Used to convert byte stream back into object
- Works with classes that implement `Serializable`
- Reads objects from file or stream

Common Methods:

- `readObject()` – reads object from stream
- `close()` – closes the stream

Example:

```
import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }
}

class Demo {
    public static void main(String args[]) throws Exception {
        ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("test.txt"));

        Student s = (Student) ois.readObject(); // reading object
        System.out.println(s.id + " " + s.name);

        ois.close();
    }
}
```

56. Java Deserialization

Deserialization is the process of converting a byte stream back into an object. It is the reverse of serialization.

- Used to retrieve object data from file or network
- Requires the class to implement `Serializable`
- Performed using `ObjectInputStream`

Steps:

- Create `ObjectInputStream`
- Read object using `readObject()`
- Typecast to original class

Example:

```
import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }
}

class Demo {
    public static void main(String args[]) throws Exception {
        ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("test.txt"));

        Student s = (Student) ois.readObject(); // deserialization
        System.out.println(s.id + " " + s.name);

        ois.close();
    }
}
```